

Full Marks:50

SET:A

Time: 1Hour 10Minutes

**Q1. What will be the output?**

```
x = 10
```

```
def fun():
    print(x)
    x = 20
```

```
fun()
```

- A)10
- B)20
- C)UnboundLocalError
- D)NameError

**Q2. What will be printed?**

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        print(x)
```

```
    x = 30
    inner()
```

```
outer()
```

- A>('10', False)
- B>('20', False)
- C>('30', True)
- D>('NameError', False)

**Q3. What is the output?**

```
def fun(x):
    x += 10
    return x
```

```
x = 5
```

```
y = fun(x)
```

```
print(x, y)
```

- A>('15 15', False)

```
B>('5 15 ', True)
```

```
C>('5 5 ', False)
```

```
D>('15 5', False)
```

**Q4. What will happen?**

```
def fun(a, b=10):
    return a + b
```

```
print(fun(b=5, a=20))
```

- A>('25', True)
- B>('15', False)
- C>('TypeError', False)
- D>('SyntaxError', False)

**Q5. What is the output?**

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

- A>('10', True)
- B>('10', False)
- C>('30', False)
- D>('20', False)

**Q6. What happens when this function is called?**

```
def fun(a=10, b):
    return a + b
```

- A('It executes correctly', False)
- B('TypeError', False)
- C('SyntaxError', True)
- D('NameError', False)

**Q7. What will be printed?**

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
```

```
b = fun()
```

```
print(a)
```



**print(b)**

- A)('10'\n[10]', False)
- B)('10'\n[10, 10]', True)
- C)('10, 10'\n[10]', False)
- D)('Error', False)

**Q8. What is the output?**

**def fun(x):**

**return x \* 2**

**a = fun**

**print(a(5))**

- A)('5', False)
- B)('10', True)
- C)('func(5)', False)
- D)('Error', False)

**Q9. What will be printed?**

**def fun():**

**print("A")**

**return**

**print("B")**

**fun()**

- A)('A', True)
- B)('B', False)
- C)('A B', False)
- D)('Nothing', False)

**Q10. What does the following function return?**

**def fun():**

**print("Hello")**

**x = fun()**

**print(x)**

- A)('Hello', False)
- B)('None', True)
- C)('Hello None', False)
- D)('Error', False)

**Q11. What is the output?**

**def outer():**

**x = 10**

**def inner():**

**nonlocal x**

**x = x + 5**

**return x**

**inner()**

**return x**

**print(outer())**

- A)('10', False)
- B)('15', True)
- C)('5', False)
- D)('UnboundLocalError', False)

**Q12. What will happen?**

**def outer():**

**x = 10**

**def inner():**

**nonlocal y**

**y = 20**

**inner()**

**outer()**

- A)('20', False)
- B)('10', False)
- C)('SyntaxError', True)
- D)('NameError', False)

**Q13. What is the output?**

**def fun(x):**

**if x == 0:**

**return**

**return x + fun(x - 1)**

**print(fun(3))**

- A)('3', False)
- B)('5', False)
- C)('6', True)
- D)('None', False)

**Q14. What will happen?**

**def fun(n):**

**if n == 0:**

**return 1**

**return n \* fun(n)**

**print(fun(3))**

- A)('6', False)



B>('3', False)  
C>('RecursionError', True)  
D>('TypeError', False)  
**Q15. What is the output?**

```
f = lambda x: x + 5
```

```
print(f(10))
```

A>('10', False)  
B>('15', True)  
C>('5', False)  
D>('Error', False)

**Q16. What will be printed?**

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A>('10', False)  
B>('20', True)  
C>('None', False)  
D>('NameError', False)

**Q17. What is the output?**

```
def outer():  
    x = 10
```

```
    def inner():  
        return x + 5
```

```
    return inner
```

```
f = outer()  
print(f())
```

A>('10', False)  
B>('15', True)  
C>('None', False)  
D>('NameError', False)

**Q18. What happens here?**

```
def fun(a):  
    return a
```

```
print(fun())  
A>('None', False)
```

B>('0', False)  
C>('TypeError', True)  
D>('NameError', False)  
**Q19. What will be printed?**

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

A>('30', True)  
B>('20', False)  
C>('TypeError', False)  
D>('SyntaxError', False)

**Q20. What happens?**

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A>('30 ', False)  
B>('20', False)  
C>('TypeError', True)  
D>('SyntaxError', False)

**Q21. Suppose sample.py contains:**

```
x = 10
```

```
def display():  
    print("Hello")
```

**What will this print?**

```
import sample
```

```
print(sample.x)  
sample.display()
```

A>('10\nHello', True)  
B>('x\ndisplay', False)  
C>('Error', False)  
D>('Only Hello', False)

**Q22. What is the major difference between the following?**

```
import math
```

```
and
```

```
from math import sqrt
```



- A)('Both import only sqrt', False)
- B)('First provides the module namespace; second imports sqrt directly', True)
- C)('First imports only sqrt', False)
- D)('There is no difference', False)

**Q23. What will be the output?**

**import math**

**print(math.sqrt(25))**

- A)('25', False)
- B)('5', False)
- C)('5.0', True)
- D)('sqrt(25)', False)

**Q24. What happens here?**

**from math import sqrt**

**print(sqrt(36))**

- A)('6', False)
- B)('6.0', True)
- C)('36', False)
- D)('NameError', False)

**Q25. Suppose test.py contains:**

**print("A")**

**if \_\_name\_\_ == "\_\_main\_\_":**  
**print("B")**

**If another Python program executes:**

**import test**

**what will normally be printed?**

- A)('A', True)
- B)('B', False)
- C)('A and B', False)
- D)('Nothing', False)

**Q26. if test.py has**  
**print("A")**

**if \_\_name\_\_ == "\_\_main\_\_":**  
**print("B")**

**If test.py is executed directly, what will be printed?**

- A)('A', False)
- B)('B', False)

C)('A followed by B', True)

D)('Nothing', False)

**Q27. What is the purpose of this statement?**

**if \_\_name\_\_ == "\_\_main\_\_":**

- A)('It prevents all imports', False)
- B)('It identifies code intended to run when the file is executed directly', True)
- C)('It creates the main function', False)
- D)('It prevents syntax errors', False)

**Q28. What will normally happen?**

**import math**

**import math**

- A)('The module is normally loaded and executed twice', False)
- B)('The module is normally loaded once and reused', True)
- C)('ImportError', False)
- D)('SyntaxError', False)

**Q29. Which object keeps track of modules that have already been imported during the current Python process?**

- A)('sys.path', False)
- B)('sys.modules', True)
- C)('sys.imports', False)
- D)('\_\_module\_\_', False)

**Q30. Suppose the current directory contains a file named random.py. What potential problem can occur with:**

**import random**

- A)('Python will always reject it', False)
- B)('The local file can shadow the intended standard-library module', True)
- C)('random cannot be imported', False)
- D)('Python automatically renames the local file', False)

**Q31. What will be printed?**

**try:**  
**print(10 / 0)**  
**except ZeroDivisionError:**  
**print("A")**  
**except Exception:**



```
print("B")
```

- A>('A', True)
- B>('B', False)
- C>('A B', False)
- D>('Nothing', False)

**Q32. What is the output?**

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

- A>('A', True)
- B>('B', False)
- C>('A B', False)
- D>('SyntaxError', False)

**Q33. Why is**

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

**codes's second handler never reached?**

- A>('ZeroDivisionError is not an exception', False)
- B>('Exception is a superclass of ZeroDivisionError', True)
- C>('except blocks execute randomly', False)
- D>('ZeroDivisionError occurs before Exception', False)

**Q34. What will be printed?**

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

- A>('A', False)
- B>('B', False)
- C>('B C', True)
- D>('A C', False)

**Q35. What is the output?**

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

- A>('A', False)
- B>('B', False)
- C>('A C', True)
- D>('B C', False)

**Q36. What happens here?**

```
try:
```

```
    raise ValueError("Wrong value")
```

```
except ValueError as e:
```

```
    print(e)
```

- A>('ValueError', False)
- B>('Wrong Value', True)
- C>('e', False)
- D>('Nothing ', False)

**Q37. What is the output?**

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

- A>('A, and execution ends normally', False)
- B>('A, followed by propagation of the same exception', True)
- C>('B', False)
- D>('No Output', False)

**Q38. What will be printed?**

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

- A>('10 only ', False)
- B>('Finally only', False)
- C>('Finally followed by 10', True)



D>('10 followed by Finally', False)

**Q39. What is the output?**

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

**print(fun())**

A>('10', False)

B>('20', True)

C>('None', False)

D>('RuntimeError', False)

**Q40. What happens here?**

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)  
A('Exception', False)  
B('MyError', True)  
C('Something went wrong', False)  
D('TypeError', False)
```

**Q41. What will be the output?**

**x = 10**

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)  
A('20\n30', True)
```

B('30\n30', False)

C('20\n10', False)

D('30\n20', False)

**Q42. What will be printed?**

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```

A('1\n2\n1\n3', True)

B('1\n2\n3\n4', False)

C('0\n1\n0\n2', False)

D('1\n1\n1\n1', False)

**Q43. What will be the output?**

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

**print(fun(-5))**

A('cleanup\n-5', False)

B('negative\ncleanup', False)

C('cleanup\n100', True)

D('100\ncleanup', False)

**Q44. What happens when this code runs?**

```
def first():  
    second()
```



```
def second():
    third()

def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
A)('NameError', False)
B)('RuntimeError cannot propagate
through functions', False)
C)('Handled', True)
D)('TypeError', False)
```

**Q45. What is the output?**

```
numbers = [1, 2, 3, 4, 5]

result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)
```

```
print(result)
A)('[2, 4]', False)
B)('[4, 8]', True)
C)('[2, 4, 6, 8, 10]', False)
D)('[1, 3, 5]', False)
```

**Q46. What will be printed?**

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
A)('10', False)
B)('24', True)
C)('9', False)
```

D)('0', False)

**Q47. Suppose module\_a.py contains:**

```
print("Module loaded")
```

```
value = 10
```

**And another file contains:**

```
import module_a
import module_a
```

```
print(module_a.value)
```

**Assuming both imports occur in the same Python process, what is normally printed?**

```
A)('Module loaded\nModule loaded\n10',
False)
B)('Module loaded\n10', True)
C)('10\n10', False)
D)('ImportError', False)
```

**Q48. What happens when this code executes?**

```
def fun():
    print(variable)
    variable = 50
```

```
variable = 10
```

```
fun()
```

```
A)('10', False)
B)('50', False)
C)('NameError', False)
D)('UnboundLocalError', True)
```

**Q49. What will be printed?**

```
class InvalidAgeError(Exception):
    pass
```

```
def check_age(age):
    if age < 18:
        raise InvalidAgeError("Age is
too low")
    return "Allowed"
```



**try:**

**print(check\_age(15))**

**except InvalidAgeError as e:**

**print(type(e).\_\_name\_\_)**

**print(e)**

A)('Exception\nAge is too low', False)

B)('InvalidAgeError\nAge is too low', True)

C)('InvalidAgeError\n15', False)

D)('Nothing, because custom exceptions cannot be caught', False)

**Q50. What is the output?**

**def fun():**

**try:**

**print("A")**

**raise ValueError**

**except ValueError:**

**print("B")**

**return 10**

**else:**

**print("C")**

**return 20**

**finally:**

**print("D")**

**print(fun())**

A)('A\nB\nD\n10', True)

B)('A\nB\nC\nD\n10', False)

C)('A\nC\nD\n20', False)

D)('A\nB\n10\nD', False)

